

Absolutely. 👍 We'll stop the **Spring Boot session for today** here. You made good progress, especially because you didn't just copy the JWT code—you answered the concepts along the way.

Spring Boot — Day 3 Complete Interview Notes

Date: 9 September 2026

Today's work continued your existing springboot-training Staff REST API. Your earlier project foundation already covered the Controller → Service → Repository → JPA/Hibernate → MySQL flow.

1. Spring Security

What is Spring Security?

Spring Security is a framework used to handle authentication and authorization in Spring applications.

It answers two important questions:

Authentication → Who are you?

Authorization → What are you allowed to do?

2. Authentication

Definition

Authentication verifies the identity of a user.

For example:

Username + Password

↓

Spring Security

↓

Verify credentials

↓
Authenticated ☒

Interview answer

“Authentication is the process of verifying the identity of a user, usually by checking their credentials such as username and password.”

3. Authorization

Definition

Authorization determines which resources and operations an authenticated user is allowed to access.

Example:

Manager
├ View Staff ☒
☒lit Staff ☐
└ Delete Staff ☒

Staff
☒ew own profile ☐
└ Delete Staff ☒

Interview answer

“Authorization determines which resources and operations an authenticated user is allowed to access.”

4. Authentication vs Authorization

Authentication	Authorization
Verifies identity	Determines permissions
WHO are you?	WHAT can you access?
Happens before authorization	Uses authenticated user's identity/roles

Authentication	Authorization
Example: username/password	Example: Manager can delete staff

Interview answer

“Authentication verifies the identity of the user, while authorization determines which resources and operations the authenticated user is allowed to access.”

5. Password Security

We learned that passwords should **never be stored as plain text**.

 Bad:

```
username: aniket
password: aniket123
```

 Good:

```
username: aniket
password: $2a$10$.....
```

The second value is a **BCrypt hash**.

6. Hashing vs Encryption

Hashing

Hashing is generally a **one-way process**.

```
Password
↓
Hash
```

You don't decrypt a password hash to get the original password.

Encryption

Encryption is designed to be **reversible** using a key.

Original data
↓
Encryption
↓
Encrypted data
↓
Decryption
↓
Original data

Interview answer

“Hashing is a one-way process that converts a password into a hashed value, while encryption is a reversible process where encrypted data can be decrypted using a key. Passwords should generally be stored using password hashing rather than encryption.”

7. BCrypt

What is BCrypt?

BCrypt is a password-hashing algorithm used to securely store passwords.

Spring Security provides:

PasswordEncoder
↑
|
BCryptPasswordEncoder

PasswordEncoder is an interface, and BCryptPasswordEncoder is an implementation.

Interview answer

“BCrypt is a password-hashing algorithm used to securely store passwords. In Spring Security, BCryptPasswordEncoder is an implementation of the PasswordEncoder interface.”

8. PasswordEncoder

We created:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

What does PasswordEncoder do?

It is used to:

- Encode/hash passwords.
- Verify passwords using matches().

Registration

```
String hashedPassword =
    passwordEncoder.encode(user.getPassword());
```

```
user.setPassword(hashedPassword);
```

Flow:

```
"aniket123"
  ↓
PasswordEncoder
  ↓
BCrypt
  ↓
BCrypt hash
  ↓
Database
```

Interview answer

“PasswordEncoder is used in Spring Security to securely hash passwords before storing them in the database and to verify passwords during authentication.”

9. Why @Bean?

We learned:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Interview answer

“@Bean tells Spring to create and manage the returned object in the Spring container so it can be injected wherever it is required.”

10. Salt

A salt is a random value used during password hashing.

The important result:

Same password

↓

Different salt

↓

Different hash

Example:

aniket123 + Salt A → Hash A

aniket123 + Salt B → Hash B

Interview answer

“A salt is a random value added during password hashing. It ensures that the same password can produce different hashes and makes attacks using precomputed hash tables more difficult.”

11. PasswordEncoder.matches()

We learned why we don't simply hash the password again and compare strings.

Because BCrypt uses a salt:

```
aniket123 → Hash A  
aniket123 → Hash B
```

The hashes may be different.

Therefore:

```
passwordEncoder.matches(  
    enteredPassword,  
    storedHash  
);
```

is used.

Interview answer

“We use matches() because BCrypt uses a salt, so hashing the same password again may produce a different hash. matches() verifies the entered password against the stored BCrypt hash.”

12. User Entity

We created a separate User entity.

```
User  
-- id  
-- username  
-- password  
-- role
```

While:

```
Staff  
-- staff information  
-- department  
-- experience  
-- other business data
```

Why separate them?

“User represents authentication and authorization information, while Staff represents business information about the employee.”

13. UserRepository

We created:

```
public interface UserRepository
    extends JpaRepository<User, Integer> {

    Optional<User> findByUsername(String username);
}
```

Purpose

UserRepository handles database operations.

The custom method:

```
findByUsername()
```

retrieves a user from the database using their username.

Interview answer

“UserRepository is responsible for database operations, and findByUsername() allows us to retrieve a user by username during authentication.”

14. UserDetailsService

Spring Security needs a way to load user information.

We created:

CustomUserDetailsService

Architecture:

Spring Security

↓

UserDetailsService

↓

UserRepository

↓
MySQL
↓
User

Interview answer

“UserDetailsService is used by Spring Security to load a user’s authentication information, usually by username. Our custom implementation uses UserRepository to retrieve the user from the database.”

15. AuthenticationManager

We configured:

```
@Bean
public AuthenticationManager authenticationManager(
    AuthenticationConfiguration config)
    throws Exception {
    return config.getAuthenticationManager();
}
```

Purpose

It coordinates the authentication process.

Username + Password
↓
AuthenticationManager
↓
UserDetailsService
↓
UserRepository
↓
PasswordEncoder
↓
Authentication result

Interview answer

“AuthenticationManager is responsible for performing the authentication process. It verifies the user’s credentials and returns an authenticated result if the credentials are valid.”

16. Login API

We created:

POST /auth/login

Request:

```
{  
  "username": "aniket",  
  "password": "aniket123"  
}
```

Flow:

```
Client  
↓  
POST /auth/login  
↓  
AuthController  
↓  
AuthenticationManager  
↓  
UserDetailsService  
↓  
UserRepository  
↓  
PasswordEncoder  
↓  
BCrypt verification  
↓  
Authentication successful
```

17. JWT

What is JWT?

JWT = JSON Web Token

JWT is used after successful authentication to represent the authenticated user and carry claims needed by the application.

It allows the client to send a token with subsequent requests instead of repeatedly sending the password.

Interview answer

“JWT is a token used to represent an authenticated user’s information. After successful login, the server generates a JWT and sends it to the client. The client sends the JWT with subsequent requests to protected APIs.”

18. JWT Structure

JWT contains three parts:

HEADER.PAYLOAD.SIGNATURE

Header

Contains information such as the signing algorithm.

Payload

Contains claims.

Our JWT contains the username as the subject:

.subject(username)

It also contains:

iat → issued-at

exp → expiration

Signature

Used to verify the token's integrity and authenticity.

19. JWT Signature

We learned:

`.signWith(...)`

creates the JWT signature using the signing key.

Interview answer

“The JWT signature is used to verify the authenticity and integrity of the token. It helps ensure that the JWT was signed by a trusted party and has not been modified.”

20. JWT Expiration

We used:

`.expiration(...)`

This specifies when the JWT expires.

Example:

JWT created

↓

Valid for 1 hour

↓

Expires

↓

Cannot be used for protected API access ❌

Interview answer

“A JWT has an expiration time so that it remains valid only for a limited period. This improves security because a stolen token cannot be used indefinitely.”

21. Bearer Token

The client sends the JWT using the Authorization header:

Authorization: Bearer <JWT>

Bearer is the authentication scheme.

Interview answer

“Bearer is the authentication scheme used in the Authorization header. It indicates that the following value is the authentication token.”

Remember:

Bearer 

Barrier 

22. Access Token

An access token is used to access protected APIs.

Client

↓

Authorization: Bearer <Access Token>

↓

Protected API

Usually, access tokens have a relatively short lifetime.

Interview answer

“An access token is used by the client to access protected APIs. It is usually short-lived and is sent with API requests.”

23. Refresh Token

A refresh token is **not the expiration time**.

It can be used to obtain a new access token when the current access token expires.

Access Token



Expires



Refresh Token



New Access Token

Interview answer

“A refresh token is used to obtain a new access token after the access token expires, so the user does not necessarily need to log in again.”

24. Complete Login Flow ★

This is one of the most important interview answers from today's session.

User enters username + password



Login request



Spring Boot



Spring Security



AuthenticationManager



UserDetailsService



UserRepository



User found



PasswordEncoder



```
BCrypt verification
↓
Authentication successful
↓
Generate JWT
↓
JWT → Client
↓
Client sends Bearer JWT
with protected API requests
```

Interview answer

“First, the user enters their username and password and sends a login request. Spring Security authenticates the user, and PasswordEncoder verifies the entered password against the stored BCrypt hash. If authentication is successful, the server generates a JWT and sends it to the client. The client then sends the JWT in the Authorization header with subsequent requests to protected APIs.”

25. JWT Filter

We started creating:

```
JwtAuthenticationFilter
```

Its responsibility is to process the JWT from incoming requests.

Flow:

```
Request
↓
Authorization Header
↓
Bearer JWT
↓
JWT Filter
↓
Extract JWT
↓
Extract username
↓
```

```
Validate JWT
↓
Create Authentication
↓
SecurityContext
↓
Controller
```

Important

We **started this part but did not finish/test it today.**

26. SecurityContext

After successful JWT validation:

```
JWT
↓
Username
↓
Authentication object
↓
SecurityContext
```

The SecurityContext allows Spring Security to know:

“This request is authenticated as this user.”

This is different from authorization.

Authentication → Who is this?

Authorization → What can they do?

27. Stateless Authentication

We learned:

SessionCreationPolicy.**STATELESS**

With JWT authentication, the client sends the token with each protected request rather than relying on a server-side authentication session.

Conceptually:

Request 1 → JWT

Request 2 → JWT

Request 3 → JWT

28. SecurityConfig

Our configuration currently includes the important concepts:

[@Configuration](#)

Configuration class.

[@Bean](#)

Creates Spring-managed objects.

PasswordEncoder

BCrypt password hashing.

AuthenticationManager

Authentication process.

SecurityFilterChain

Security rules for HTTP requests.

SessionCreationPolicy.[STATELESS](#)

JWT-style stateless authentication.

29. What We Actually Implemented Today

Your project has progressed from:

Staff REST API

to:

Staff REST API

+

Validation

+

Exception Handling

+

Spring Security

+

User

+

UserRepository

+

BCrypt

+

PasswordEncoder

+

UserDetailsService

+

AuthenticationManager

+

Register API

+

Login API

+

JWT generation

And you successfully tested:

Registration → BCrypt hash → MySQL ✓

Login → Authentication → 200 OK ✓

Login → JWT generation → JWT received ✓

🔥 That's a substantial Day 3 session.



What remains

We will **NOT** continue this today.

Tomorrow we'll finish the remaining security implementation:

1. JWT Authentication Filter

Bearer JWT

↓

Filter

↓

Validate JWT

↓

SecurityContext

2. Protect Staff APIs

GET /api/staff

POST /api/staff

PUT /api/staff/{id}

DELETE /api/staff/{id}

Test:

No JWT → ❌

Valid JWT → ✅

Invalid JWT → ❌

Expired JWT → ❌

3. Roles & Authorization

Implement:

MANAGER

STAFF

Example:

MANAGER → edit/delete staff ✅

STAFF → edit/delete staff ❌

4. Method/URL Authorization

We'll learn how Spring Security actually applies role-based restrictions.

5. Complete Security Testing

We'll test the complete flow in Postman.

6. Final Interview Revision

I'll ask you questions without giving you the answers first, including:

- Spring Security
 - Authentication vs Authorization
 - BCrypt
 - PasswordEncoder
 - UserDetailsService
 - AuthenticationManager
 - JWT
 - JWT signature
 - JWT expiration
 - Bearer token
 - JWT filter
 - SecurityContext
 - Roles
-



Tomorrow's Spring Boot Topic

Spring Security — JWT Authentication + Role-Based Authorization

JWT Filter



JWT Validation



SecurityContext



Protected APIs



Roles



MANAGER / STAFF



Authorization



Complete Security

Tomorrow's goal: finish Spring Security/JWT in your existing springboot-training project and make the Staff APIs genuinely protected by JWT and roles.

For today, **stop here.** 

You have completed the **conceptual foundation + registration + BCrypt + authentication + login + JWT generation** portion. Tomorrow we pick up exactly from **JwtAuthenticationFilter**, not from the beginning.